The Cauldron

Full-Protocol Security Audit

a from-scratch review of the eternal-token machine

Subject: MagicFrens "Cauldron" — Uniswap v4 hook-native, self-relaunching token machine, with the new *prime-buy*, *proposer-incentive* and *V2-upgrade (modular controller)* subsystems.

Review type: Full re-audit (all findings re-derived; no prior report reused).

Network: Ethereum Sepolia (test) → mainnet-candidate.

Date: 31 August 2026.

Contents

1 Abstract

The Cauldron is a single, permissionless, self-perpetuating token machine built as a Uniswap v4 hook. One "iteration" lives as a v4 pool whose entire supply is held as liquidity: a thin *active* band that sets price, and a large *out-of-range reserve* that backs a redemption floor and funds 1:1 migration. When an iteration's rolling volume dies, anyone may `relaunch()`: the dead LP is recovered, its tokens burned, and a fresh iteration is born from a governor-selected community proposal — seeded with a real first-block "green-candle" market buy. Genesis MiFrens holders earn a permanent founder's cut of every iteration; per-collection "legacy" floors and an in-hook perpetuals engine layer on top.

This review is a *from-scratch* re-derivation of the protocol's security posture, undertaken before the next deployment ("r30"). It covers the full contract set and, in particular, three subsystems added since the last review:

1. **Prime buy** — an owner-funded, personal-ETH first-block market buy at genesis whose output goes to the treasury for a later OG airdrop (net demand, zero supply dilution).
2. **Proposer incentive** — a tiny per-iteration fee slice that rewards whoever authored the winning proposal (a decentralization flywheel).
3. **Modular V2-upgrade path** — a governed controller handoff that lets a future "cellular division" V2 inherit the live liquidity *by ownership transfer, without a teardown*, protected by a hard exit guarantee and a guardian veto.

One **Medium** issue was found *and fixed during this review* (A-01: an untrusted external call in the swap fee hot path). No unresolved Critical or High severity issue that risks principal was identified. The dominant residual risk is *governance centralization*, which the protocol addresses by an explicit, deliberately-chosen "powerful but telegraphed, with a guaranteed floor exit" trust model rather than by hard on-chain constraints (§6.2).

2 Methodology

The review followed a manual, adversarial reading of every state-changing external entry point, cross-referenced against automated size/compile checks and the project's own Foundry fork-test suite (141 tests against live Sepolia v4). For each entry point we asked: who may call it; what value can move; is Checks-Effects-Interactions respected; is any external call made to an untrusted address; and can a swap, relaunch, or redemption be bricked or front-run. Findings were classified by the standard matrix:

Critical	Direct principal loss, no special conditions.
High	Principal loss under specific but reachable conditions.
Medium	Limited loss, griefing, or a governance/centralization lever.
Low	Minor / best-practice / defence-in-depth.
Info	Observation, or a positive design confirmation.

3 Scope

Contract	Role
`CauldronRegistry`	Orchestrator: summon, relaunch, redemption floor, prime buy, emergency & V2-upgrade governance. Custody of the live position-NFTs.

Contract	Role
`CauldronHook`	v4 hook: swap fees, anti-snipe surtax, volume/death, legacy buy-back, guild & proposer routing, gacha, perp-fee routing.
`PoolOps` (library)	Delegatecalled LP primitives: seed, green-candle buy, prime buy, reserve add/claim, migration, `CauldronToken` CREATE2 deploy.
`CauldronToken`	Fixed-supply, non-mintable, non-freezable ERC-20 per iteration.
`PerpEngine` / `PerpVault`	Hook-native perpetuals + community PLV vault.
`CollectionLedger`	Per-collection legacy-floor cap table (accounting only).
`MiFrensGenesis` / `MiFrensDividend`	OG NFT + founder fee dividend.
`CauldronGovernor` / `Factory` / `GachaRouter`	Proposals, collection deploy, crystal gacha router.
`ReserveLib` / `LaunchLib`	Pure tick / naming helpers.

Timelock: OpenZeppelin `TimelockController` is the registry's `emergencyAdmin` and the owner of the hook & perp engine post-deploy.

4 Access-Control & Fund-Mover Matrix

Every function that can move ETH or tokens *out* of the system, and its gate:

Fund-mover	Caller gate	Extra guard
`emergencyWithdrawLP`	emergencyAdmin (timelock)	armed+delay, exit-open, vetoable, nonReentrant
`emergencySweep`	emergencyAdmin (timelock)	armed+delay, vetoable, nonReentrant
`migrateToSuccessor`	emergencyAdmin (timelock)	armed+delay, exit-open, vetoable, nonReentrant
`redeemFren / claimByBurn`	any NFT owner / holder	bounded by floor; paid from reserve LP
`buyTreasuryFren / recycle`	permissionless	pays/pulls at the accounted floor
`sweepPrimeBuy`	primeFunder	CEI (balance zeroed first)
`releaseRelaunchETH`	registry only	sends tracked `relaunchETH` var
`claimProposerFees`	self (pull)	CEI + nonReentrant
`PerpVault / PerpEngine.withdrawPlvTo`	vault / share-owner	onlyVault, notNested, nonReentrant

No path transfers principal to an arbitrary, caller-supplied address without either (a) a timelock+veto governance gate or (b) a floor-bounded, permissionless redemption. The one place an attacker-controlled address received a push — the proposer slice — was converted to a pull during this review (A-01).

5 Findings Summary

ID	Severity	Title	Status
A-01	Medium	Untrusted external call in the swap fee hot path (proposer push)	**Fixed**
A-02	Medium	Governance centralization (custody & upgrade levers)	Mitigated (by design)
A-03	Low	Hook re-point exposes the `relaunchETH` buffer	Acknowledged
A-04	Low	Unchecked ERC-20 return values on protocol tokens	Acknowledged
A-05	Info	Stale position pointers after `migrateToSuccessor`	By design
A-06	Info	Prime-buy recipient-zero fail-safe	By design
A-07	Positive	Successor handoff preserves price & liquidity (no teardown)	Confirmed
A-08	Positive	Exit guarantee: redemptions forced open while armed	Confirmed
A-09	Positive	Reserve solvency invariant (fuzz + fork proven)	Confirmed

6 Detailed Findings

6.1 A-01 — Untrusted external call in the swap fee hot path (Medium, Fixed)

A-01 · CauldronHook · Reentrancy / Untrusted Call

The proposer-incentive slice was originally *pushed* to `activeProposer` with a value-bearing `.call` inside `_routeEthFee`, which runs during `beforeSwap`/`afterSwap`.

Impact. `activeProposer` is attacker-controllable: anyone may submit a proposal, so a winning proposer can point the slice at a contract of their choosing. A value push to that address inside the un-guarded hook swap entrypoint is a classic reentrancy surface — the recipient executes mid-swap, before the fee's guild/floor/relaunch state updates complete, and could attempt to re-enter the hook, registry, or a nested swap.

Resolution (this review). Converted to a **pull pattern**. The slice now accrues to `proposerOwed[proposer]` (ETH retained in the hook, tracked by variable exactly like `relaunchETH`/`legacyBuffer`), and the proposer later withdraws via `claimProposerFees()` (CEI + `nonReentrant`). The swap hot path now performs *zero* external calls to untrusted addresses.

```
// before (push, untrusted call mid-swap):
(bool okP,) = prop.call{value: wantProp}("");
// after (pull, no call in the hot path):
proposerOwed[prop] += wantProp; feeAmount -= wantProp; emit ProposerFunded(
    prop, wantProp);
```

Verified by `test_ProposerIncentive_PaysProposer_OnFork` (accrues, then claims exactly, then zeroes).

6.2 A-02 — Governance centralization of custody & upgrade levers (Medium, mitigated by design)

A-02 · CauldronRegistry / CauldronHook · Centralization

`emergencyWithdrawLP`, `emergencySweep`, `migrateToSuccessor` and the hook's `setRegistryOverride` let the governance timelock move the live liquidity or re-point the controller. This is the protocol's largest trust surface.

Design posture. The project deliberately chooses a *"powerful but telegraphed, with a guaranteed exit"* model over hard "cannot-rug-by-construction" constraints, in order to keep a break-glass escape and to leave a future V2 unconstrained. The residual risk is bounded by **five independent layers**:

1. **Timelock delay** — every custody action must be `armEmergency`'d and wait `emergencyDelay` (mainnet: days), with a public event.
2. **Forced-open exit** — the instant any action is armed, `_redeemBlocked()` returns false regardless of `redemptionPaused`; holders can always redeem at floor *before* anything moves. This closes the "pause-then-drain" trap.
3. **Guardian veto** — a block-only `guardian` may `vetoEmergency()` to cancel an armed action; it can never propose or move funds.
4. **Reserve solvency** — the exit is *funded*: the out-of-range reserve is intact during the whole window (the withdraw is the last step), so a bank-run at floor is payable (A-09).
5. **Community ratification (recommended for mainnet)** — gate `setSuccessor` behind a genesis-fren vote so one key cannot hand off.

Residual. Layers 1–4 protect *attentive* holders and passive holders alike (the veto is passive-safe). A fully-compromised timelock *and* guardian could still, after the delay, execute a withdraw — but holders retained a funded floor exit throughout. Recommendation: a multi-sig guardian distinct from the timelock proposer set, and a $\geq$7-day `emergencyDelay` on mainnet.

6.3 A-03 — Hook re-point exposes the `relaunchETH` buffer (Low)

A-03 · CauldronHook · Access Control

`setRegistryOverride` (onlyOwner = timelock) re-points the hook's trusted `registry`. A malicious re-point could let a substitute "registry" call `releaseRelaunchETH` and drain the accrued relaunch buffer.

Impact. Bounded to the `relaunchETH` buffer (fees accrued since the last relaunch) — *not* the LP, which is owned by the registry address, not the hook. Requires a compromised/malicious timelock (same base trust as A-02). **Recommendation.** Perform the hook re-point in the *same* governance ceremony as `migrateToSuccessor` (which carries the exit window), and treat it as a custody-class action operationally.

6.4 A-04 — Unchecked ERC-20 return values (Low)

A-04 · Multiple · Unchecked Calls

Several `IERC20.transfer/transferFrom` calls do not check the boolean return (e.g. registry airdrop/sweep, perp token funding).

Impact. Low: the only tokens handled are the protocol's own `CauldronToken` (OpenZeppelin-based, reverts on failure) and ETH. A non-standard token is never in scope. **Recommendation.** Adopt `SafeERC20` uniformly before mainnet as defence-in-depth against any future integration of a non-reverting token.

6.5 A-05 — Stale position pointers after migration (Info, by design)

A-05 · CauldronRegistry · Logic

After `migrateToSuccessor`, `generationPositionId/...ReservePositionId` still reference NFTs the registry no longer owns; a subsequent `redeemFren` on the old registry would revert.

Rationale. Intended: the old registry is *defunct* post-handoff; all custody and user flows move to the successor. Reverting (rather than mis-paying) is the fail-safe outcome. Front-ends must route redemptions to the successor after a migration.

6.6 A-06 — Prime-buy recipient-zero fail-safe (Info, by design)

A-06 · CauldronRegistry / PoolOps · Input Validation

The genesis prime buy sends bought tokens to `airdropWallet`, falling back to `primeFunder`. If both were zero the `take` to `address(0)` would revert.

Rationale. `primeBuyEth` can only be funded by `primeFunder`, which is therefore non-zero whenever a prime buy exists — the recipient is non-zero by construction. A misconfiguration reverts the summon *loudly* rather than burning tokens. Acceptable fail-safe.

6.7 A-07 — Successor handoff preserves price & liquidity (Positive)

A-07 · CauldronRegistry · Upgradeability

`migrateToSuccessor` transfers the position-NFTs' *ownership*; it never withdraws liquidity.

Confirmation. Uniswap v4 positions are ERC-721s; transferring the token changes only the owner-of-record, leaving the pool's `sqrtPrice`, tick, and position liquidity untouched. Fork-proven byte-for-byte by `test_MigrateToSuccessor_MovesOwnership_NoTeardown_OnFork`. This is what makes a V2 controller a *drop-in swap* rather than an emergency-withdraw + redeploy.

6.8 A-08 — Exit guarantee: redemptions forced open while armed (Positive)

A-08 · CauldronRegistry · Safety

`_redeemBlocked() = redemptionPaused && emergencyReadyAt == 0`.

Confirmation. The redemption pause cannot be used to trap holders ahead of a custody move: arming any emergency action forces the floor exit open for the whole window. Fork-proven by `test_ExitForcedOpenWhileArmed_AndGuardianVeto_OnFork` (paused→blocked; armed→open; vetoed→blocked again; non-guardian cannot veto).

6.9 A-09 — Reserve solvency invariant (Positive)

A-09 · CauldronRegistry / CollectionLedger · Economic

The out-of-range reserve always covers 1:1 migration + unclaimed genesis + crystallized legacy entitlements.

Confirmation. The relaunch reserve is sized as `TOTAL_SUPPLY - newActive`, where `newActive` is the survived circulating supply net of unclaimed genesis and legacy carve. The `CollectionLedger` conservation invariant is fuzz-proven over 128,000 operations; 1:1 migration out of the freshly-bought reserve is fork-proven under legacy-carve stress.

7 Invariants & Test Coverage

- **141 / 141** Foundry tests pass (fork against live Sepolia v4).
- All in-scope deployable contracts are under the EIP-170 24,576-byte limit (registry 23.7 KB, hook 22.8 KB, engine 24.4 KB, PoolOps 17.0 KB).
- Key fork proofs: genesis prime buy (treasury receives GNOME, zero dilution, spot pumps); proposer accrual/claim; exit-guarantee & guardian veto; migration no-teardown; 1:1 migration solvency; perp auto-migrate at relaunch.

8 Gas Optimization Audit

A full-scope gas review was run across all contracts, prompted by a concrete failure: **in-swap liquidation of a short position silently no-oped under a normal swap's gas budget**. A short liquidation was measured at ≈**957k gas**; the hook only forwards `gasleft − 180k` to the sweep and only fires it above ≈430k, so an ordinary router/UI swap never carried enough gas to settle a short — the position stayed open despite being underwater at the mark, and no Liquidatoor badge was struck.

ID	Contract	Optimization	Impact
G-01	PerpEngine	`OBS_CARDINALITY` 128→32 — the TWAP mark loop cold-read 128 slots but a 5-min window at 30s spacing can never use >∼10	≈−200k on the first mark loop
G-02	PerpEngine	Cache the TWAP mark ONCE per liquidation — it was recomputed 2–3× (`_underwater` + notional-cap); now the value is computed once and reused (`_underwaterVal`)	removes 1–2 redundant mark loops
G-03	PerpEngine	Liquidatoor badge is now **hybrid**: a gas-bounded best-effort auto-mint in-swap (matching the creature-NFT UX) that downgrades to a claimable credit (`claimLiquidatorBadges`) only when gas is tight — so a liquidation never reverts on the trophy	removes the badge from the critical path
G-04	All	Loop hygiene: cache `.length`, `unchecked {++i}`, and cache the `minted` counter in `MiFrensGenesis.mint` (was a storage read+write every iteration, up to 100×/mint)	per-iteration SLOAD/SSTORE savings; also *shrinks*

Result. A short liquidation dropped from ≈957k to ≈450k gas, and the in-swap path (which reuses the already-open PoolManager lock and no longer mints the badge) is lighter still — so swaps carrying a normal router/UI gas budget now trigger short liquidations, closing the "every swap liquidates" gap. Ultra-tight swaps still defer to the permissionless keeper (`liquidate`/`sweepLiquidations`), which mainnet MEV bots run. All 141 fork tests remain green and every deployable contract stays under EIP-170 (the `unchecked` loops reclaimed bytecode headroom on the byte-tight engine).

Note. These are code-level improvements; the currently-deployed r30 contracts realise them on the next deployment (r31). No behavioural or economic change — badge attribution is preserved by the event; the pull-claim mints the same trophy into the live collection.

9 Conclusion

The Cauldron's principal-holding paths are, to the depth of this review, free of unresolved Critical/High vulnerabilities. The one Medium issue surfaced by this review — an untrusted external call in the swap fee path — was fixed in place with a pull pattern. The protocol's remaining material risk is governance centralization, which it addresses not by removing power but by making every dangerous action slow, visible, vetoable, and — crucially — always preceded by a *funded, un-blockable floor exit*. For mainnet we recommend: a ≥7-day emergency delay, a guardian multisig independent of the timelock proposers, `SafeERC20` throughout, and gating `setSuccessor` behind a genesis-fren ratification vote. With those, the "eternal machine" can upgrade itself toward the V2 cellular-division design as a governed pointer swap on live liquidity — never a teardown.